

PPT-LAT-SP-MODEL — .sp-model byte layout v0

Companion document to PPT-LAT-L1-ABI-v0. Defines the on-disk byte layout for .sp-model and the paired .sp-tokenizer such that sp_model_load is implementable as open + mmap + parse header + set up tensor table pointers with zero allocation beyond the opaque sp_model handle itself. Every structural decision here is driven by the locked ABI (caller-allocates, mmap-friendly, no per-call file IO).

The Spinor 63→64 padding question — on-disk pad vs scatter-at-load — is decided in §6.

1. Goals and non-goals

Goals.

- sp_model_load is pure mmap + pointer setup. No malloc for tensor data. No memcpy of tensor data. The file IS the in-memory layout.
- Every byte the math kernels read at runtime is already at the correct alignment the moment mmap returns.
- Header parsing is a single memcpy into a fixed struct.
- Tensor lookup is $O(\log N)$ via name-hash binary search on a fixed 256-byte tensor table entry.
- The format is forward-compatible: v0 readers can refuse v1 files cleanly via the version field; v1 readers can fall back to v0 semantics by ignoring reserved bytes.
- Disk overhead is $\leq 2\%$ vs the raw packed tensor data. Spinor padding is the dominant contributor at $\sim 1.5\%$.

Non-goals.

- Compression on disk. PPT weights are already compressed (Q8/Q4/Spinor); zstd-on-top buys little and breaks mmap.
- Encryption. Use OS-level (filesystem) encryption if needed.
- Multi-version-in-one-file. Each .sp-model is one model snapshot.
- Streaming load from network. Caller mmmaps a local file; remote-fetch is L2/L3's concern.
- Tokenizer content. That's .sp-tokenizer's file; here we only carry its SHA-256.

2. Byte order, alignment, file extension

Endianness. All multi-byte integers and floats are little-endian. Target hardware (x86_64, ARM64, CUDA hosts, Hexagon V69) is uniformly little-endian in practice. Big-endian platforms must byte-swap at load or refuse the file. We do not pay portability cost for systems we will not ship to.

Alignment.

- **Header** lives at file offset 0, fixed 512 bytes (one sector).
- **Tensor table** is 64-byte-aligned, fixed at file offset 512.
- **Tensor data region** is 65536-byte-aligned (one Windows MapViewOfFile granularity unit). Each individual tensor inside it is 64-byte-aligned.

The 65536-byte data-region alignment matters specifically on Windows: partial mmmaps (MapViewOfFile) require the base offset to be a multiple of the allocation granularity (64 KB on Windows; 4 KB on Linux). Aligning the data region to the larger constant means a single MapViewOfFile call can map any tensor independently without header-region overhead.

File extension. .sp-model for the weights + arch + tokenizer hash, .sp-tokenizer for the paired tokenizer blob. Both files together constitute a deployable PPT model.

3. File header (fixed 512 bytes)

offset	size	field	notes
0	4	magic	ASCII "SPMD" (0x44 0x4D 0x50 0x53 LE)
4	2	version_major	u16, v0 = 0
6	2	version_minor	u16, v0 = 1
8	4	header_size	u32, total bytes of this header = 512
12	4	arch_id	u32, sp_arch_info.arch_id enum (LLAMA3=1, QWEN3=2, GEMMA3=3, DEEPSEEK_V4=4, ...)
16	4	arch_struct_size	u32, in bytes; for v0, sizeof(sp_arch_info)
20	4	arch_struct_capacity	u32, on-disk reservation = 256
24	256	arch_struct	memcpy-direct payload for sp_arch_info; unused tail zero-filled to 256
280	32	tokenizer_hash	u8[32], SHA-256 of paired .sp-tokenizer (entire .sp-tokenizer file, byte-by-byte)
312	4	vocab_size	u32, mirrors arch_struct field for fast pre-allocation; loader asserts equality
316	4	tensor_count	u32, number of entries in tensor table
320	8	tensor_table_offset	u64, byte offset of first tensor entry (= 512 in v0)
328	8	tensor_data_offset	u64, byte offset of first tensor byte (multiple of 65536)
336	8	file_size	u64, total file size in bytes; loader asserts == stat(file).st_size
344	8	created_unix_seconds	u64, wall-clock seconds since epoch at transcode time
352	8	transcoded_from	u64, hash of upstream file path (e.g. fxhash of the GGUF source path); zero if model was generated natively
360	4	header_crc32	u32, CRC-32 of bytes [0, 360) using standard IEEE polynomial; the field itself is excluded from coverage
364	148	reserved	zero-filled in v0
512	...	--	tensor table starts here

All offsets and sizes are absolute byte offsets from file start.

Why header_size is explicit and the reserved tail exists. v1 may extend the header by reusing reserved bytes. v0 readers memcpy(512 bytes) and ignore tail-extension fields they don't recognize. v1 readers compare header_size == 512 and run v0-compatible parsing on match, v1 parsing on mismatch (which implies new fields after byte 512). This is forward-compatible without a parser dispatch table.

Why CRC-32 of the header only. Per-tensor integrity is covered by the BLAKE3 hash in each tensor entry (§4). The header CRC catches gross corruption of the header itself (page tear, partial write during transcode) before the loader tries to interpret tensor offsets.

4. Tensor table (256-byte entries)

Tensor table entries are fixed-size so the table is a flat array addressable as `entry[i] = tensor_table_offset + i * 256`. No string table, no variable-length records, no second-level indirection.

```
typedef struct {
    char    name[80];           // bytes 0-79: null-terminated tensor name
```

```

uint32_t dtype_id;           // (longest GGUF names are ~30 chars; 80 is
uint32_t n_dims;             // bytes 80-83: enum, see §5
uint64_t dims[8];            // bytes 84-87: rank, 1..8
                               // bytes 88-151: shape in elements (not bytes);
                               // unused entries are zero, NOT one
uint64_t offset_in_data;     // bytes 152-159: byte offset from tensor_data_offset;
                               // multiple of 64
uint64_t size_bytes;         // bytes 160-167: on-disk byte length, including
                               // any per-block padding (Spinor +1)
uint32_t block_size;         // bytes 168-171: on-disk granularity (64 for Spinor,
                               // 32 for Q4 row-block, 1 for fp16/fp32)
uint32_t block_count;        // bytes 172-175: size_bytes / block_size, sanity
uint8_t blake3[32];          // bytes 176-207: BLAKE3-256 of the tensor's
                               // on-disk bytes (size_bytes long,
                               // starting at tensor_data_offset +
                               // offset_in_data)
uint64_t name_hash;          // bytes 208-215: xxh3_64(name); table is sorted
                               // by this for binary lookup
uint8_t reserved[40];        // bytes 216-255: zero-filled in v0
} sp_tensor_entry;
// sizeof = 256

```

Entries are sorted by name_hash ascending. Loader binary-searches by hash, then verifies name equality on the match (defends against the 1-in- 2^{64} hash collision). Sort + hash means lookup is $O(\log N + 1 \text{ strcmp})$, which on a 2000-tensor model is ~11 hash comparisons and one string compare.

Why name is fixed at 80 bytes. GGUF's longest tensor names (blk.NN.ffn_gate_inps.weight and variants) fit in 40 bytes; 80 gives headroom for compound names from MoE / SSM / hybrid architectures without going variable-length.

Why dims is u64. Allows >4G-element tensors without ambiguity. For a vocab projection on a 32B model, `dims[0] * dims[1]` already exceeds 2^{32} .

Why per-tensor BLAKE3 rather than CRC. BLAKE3 is fast on modern hardware (~5 GB/s/core), cryptographically strong, and catches the specific failure modes we care about (bit-rot, partial writes, model provenance). Verifying every tensor on load is opt-in via `sp_session_config.flags & SP_VERIFY_TENSORS`. The default load trusts the file and just checks the header CRC.

5. dtype_id enum

```

typedef enum {
    // Continuous (parity with GGUF for direct transcode of unquantized tensors)
    SP_DT_F32      = 1,    // 4 bytes/elem, block_size=1
    SP_DT_F16      = 2,    // 2 bytes/elem, block_size=1
    SP_DT_BF16     = 3,    // 2 bytes/elem, block_size=1

    // PPT-native quant types
    SP_DT_OK_Q8    = 10,   // 0_K-lifted int8 + per-row Frobenius scale
                               // (scale lives in paired tensor with .scale suffix)
                               // block_size=1, 1 byte/elem
    SP_DT_OK_Q4    = 11,   // 0_K-lifted int4 packed two-per-byte
                               // block_size=32, 16 bytes per 32 elements
}

```

```

SP_DT_FROBENIUS_SCALE_FP32 = 12,
                                // companion to Q8/Q4 weight tensor; one fp32 scalar
                                // per row of the weight; named "<weight>.scale"

// Discrete-algebra-native types
SP_DT_SPINOR63 = 20,           // 63-byte logical block, 64-byte on-disk block
                                // (see §6); block_size=64, block_count = N_blocks
SP_DT_RING_RESIDUE_CRT_30_30 = 30,
                                // dual-prime CRT residue pair; two u32 per element
                                // block_size=8 (one (r1, r2) pair)
SP_DT_OK_INTEGER = 31,         // 0_K[√-163] elements stored as (Re i32, Im i32)
                                // block_size=8
} sp_dtype_id;

```

dtype_id space is partitioned: 0–9 reserved continuous, 10–19 quant, 20–29 discrete-algebra block formats, 30–39 ring residues, 40+ reserved for v1+ formats (Stern-Brocot tables, factored ARM bank, etc.).

Each dtype’s relationship between logical shape (dims[]) and on-disk byte length is mechanical:

dtype	logical elements per block	bytes per block on disk
F32	1	4
F16 / BF16	1	2
OK_Q8	1	1
OK_Q4	32	16
FROBENIUS_SCALE_FP32	1	4
SPINOR63	(arch-defined; one Spinor signature)	64 (63 + 1 pad)
RING_RESIDUE_CRT_30_30	1	8
OK_INTEGER	1	8

block_size in the tensor entry is the on-disk bytes-per-block; the loader uses block_size * block_count == size_bytes as a sanity invariant.

6. The Spinor 63→64 padding decision

Decision: on-disk padding to 64 bytes per Spinor block, byte 63 of each block holds the sentinel value 0xA5.

Two alternatives were considered:

Option A — on-disk padding (chosen). Each Spinor block occupies exactly 64 bytes on disk: bytes 0..62 are the Spinor payload, byte 63 is the sentinel 0xA5. The block_size field in the tensor entry is 64. The mmap’d region presents 64-byte-aligned blocks; the loader does nothing; the file IS the in-memory layout.

Option B — scatter at load. Spinor blocks are packed contiguously on disk (each 63 bytes), and the loader scatters them into a 64-byte- aligned arena at load time. Saves 1.5% of disk per Spinor tensor.

Option A wins on three structural grounds, not just preference:

1. **The ABI requires it.** PPT-LAT-L1-ABI-v0 §2 commits L1 to a caller-allocates discipline with no L1-side malloc crossing the FFI. A scatter-at-load step would have to allocate a fresh arena inside sp_model_load, doubling RAM during load (mmap’d source + destination

arena) and adding an $O(N)$ copy. That breaks the “the file IS the load” property that justifies the format existing in the first place.

2. **Spinor blocks are read in 64-byte SIMD-friendly chunks anyway.** AVX-512 / NEON loads are 64 bytes wide. Reading a 63-byte block via `vmovdqu64` past the 63rd byte is undefined unless the byte at +63 is allocated and readable. On-disk padding makes the read well-defined; scatter-at-load forces a per-block bounds check or a slower $32+16+8+4+2+1$ fallback.
3. **The sentinel doubles as cheap integrity.** A 1.5% disk overhead is the price; in exchange, byte 63 holds `0xA5` (`0b10100101` — high bit density, unlikely to arise from zero-fill or sparse-write corruption). Any page tear, partial write, or filesystem corruption that touches the Spinor block but leaves byte 63 alone is impossible by construction. The verifier (opt-in, `SP_VERIFY_TENSORS`) scans every Spinor block’s byte 63 in a tight AVX-512 stride; a mismatch returns `SP_ESPINOR_BADBLOCK` per the ABI’s error surface (PPT-LAT-L1-ABI §7).

Disk-overhead math: a Gemma3-1B-class model with Spinor-formatted KV adjuncts carries ~50 M Spinor blocks. At 1 byte pad each, that’s 50 MB of overhead on a model whose total `.sp-model` size is ~2 GB. 2.5% of the Spinor portion, ~1.5% of total file size. Acceptable.

On-disk sentinel value `0xA5` rationale. Three constraints: (a) distinct from zero (catches zero-fill corruption), (b) distinct from the all-ones byte `0xFF` (catches “device returned -1 on read” errors that get memcpy’d in), (c) bit pattern that’s unlikely to be produced by misaligned shifts of legitimate Spinor payload bytes. `0xA5 = 0b10100101` satisfies all three.

7. Companion `.sp-tokenizer` file

offset	size	field	notes
0	4	magic	ASCII "SPTK"
4	2	version_major	<code>v0 = 0</code>
6	2	version_minor	<code>v0 = 1</code>
8	4	header_size	<code>u32</code> , <code>v0 = 128</code>
12	4	type_id	enum: <code>SENTENCEPIECE=0</code> , <code>BPE_LLAMA3=1</code> , <code>BPE_GPT2=2</code> , <code>TIKTOKEN_0200K=3</code> , ...
16	4	vocab_size	<code>u32</code> , must match <code>.sp-model.vocab_size</code>
20	4	bos_token	<code>u32</code> , token id (or <code>0xFFFFFFFF</code> if absent)
24	4	eos_token	<code>u32</code> , token id (or <code>0xFFFFFFFF</code> if absent)
28	4	pad_token	<code>u32</code> , token id (or <code>0xFFFFFFFF</code> if absent)
32	4	unk_token	<code>u32</code> , token id (or <code>0xFFFFFFFF</code> if absent)
36	8	blob_offset	<code>u64</code> , byte offset of raw tokenizer blob
44	8	blob_size	<code>u64</code> , length of raw tokenizer blob in bytes
52	4	header_crc32	<code>u32</code> , CRC-32 of bytes [0, 52)
56	72	reserved	zero-filled
128	...	blob	raw <code>SentencePiece</code> / BPE bytes, as-shipped

The blob is exactly what HuggingFace ships in `tokenizer.json` / `tokenizer.model` for the corresponding tokenizer type — we do not reformat or reparse it. L2 hands the blob to a `SentencePiece` / `tokenizers` crate at load time. L1 never touches it; L1 only verifies the SHA-256 of the entire `.sp-tokenizer` file matches the `tokenizer_hash` in the paired `.sp-model` header.

This is what makes `.sp-tokenizer` reusable across fine-tunes: a Llama-3-Instruct, Llama-3-Code, and Llama-3-Chat fine-tune all share the same `.sp-tokenizer`, while each ships its own `.sp-model`. The mismatch case (“you loaded a Qwen3 model with the Llama-3 tokenizer”) returns `SP_ETOKENIZER_HASH` per the ABI’s error surface.

8. Load procedure — `sp_model_load` reference implementation

In pseudocode (a real implementation is ~200 lines of C):

```
fn sp_model_load(model_path, tokenizer_path, out_model) -> sp_status:
  1. open(model_path), stat
  2. mmap(file_size, READ, SHARED, fd, 0)           // single syscall
  3. memcpy(&header, mmap_base, 512)
  4. verify header.magic == "SPMD"
  5. verify CRC32(mmap_base[0..360]) == header.header_crc32
  6. verify header.file_size == stat.st_size
  7. verify version_major == 0 (or dispatch v1 reader)
  8. memcpy(&model.arch, header.arch_struct, header.arch_struct_size)
  9. tensor_table_ptr = mmap_base + header.tensor_table_offset
 10. tensor_data_ptr = mmap_base + header.tensor_data_offset
 11. verify (header.tensor_table_offset % 64) == 0
 12. verify (header.tensor_data_offset % 65536) == 0
 13. open(tokenizer_path), stat
 14. compute SHA-256 of tokenizer file
 15. verify SHA-256 matches header.tokenizer_hash
      → on mismatch: return SP_ETOKENIZER_HASH
 16. mmap tokenizer file separately (smaller, often shared
      across many models)
 17. *out_model = sp_model {
      mmap_base, mmap_size,
      arch,
      tensor_table_ptr, tensor_count,
      tensor_data_ptr,
      tokenizer_mmap_base, tokenizer_blob_offset, tokenizer_blob_size,
  }
 18. return SP_OK
```

No malloc in the hot path; `sp_model` itself is a small heap-allocated struct holding pointers into the mmap regions. Total load time is dominated by SHA-256 of the tokenizer file (typically <50 ms for a 1-2 MB tokenizer) plus header CRC (microseconds). The mmap is lazy — pages fault in as tensors are first accessed, which is exactly what we want.

9. gguf-to-sp transcoder responsibilities

The transcoder is the *one-shot* path from upstream GGUF to PPT- native `.sp-model`. Run offline, once per model, on the workstation that has the source GGUF.

Per-tensor transcoding:

- **Unquantized tensors** (norms, biases, RoPE inverse-frequency tables): copied bit-for-bit. dtype stays F32 / F16.
- **GGUF quantized tensors** (Q8_0, Q4_K, etc.): dequantize to F32, then re-quantize into OK_Q8 or OK_Q4 with per-row Frobenius scale. The scale becomes a separate tensor named `<original>.scale` of dtype `FROBENIUS_SCALE_FP32`.
- **Attention / Spinor-eligible tensors**: optionally re-pack into Spinor signatures during transcode if the source arch supports it. If not, leave as F16/F32 and let the engine apply Spinor at runtime via the KV cache hook.

Arch detection: pull `general.architecture` from the GGUF metadata, map to `sp_arch_info.arch_id`. The transcoder owns the per-arch metadata extraction (RoPE base, GQA group count, SWA window, FFN variant), populates `sp_arch_info`, embeds it into the `arch_struct` field of the

header.

Tokenizer extraction: GGUF embeds the tokenizer; we strip it out and write `.sp-tokenizer`. SHA-256 of the resulting `.sp-tokenizer` goes into the `.sp-model` header.

Transcoder is a separate binary, `sp-transcode`. Not part of `libshannonprime`. Lives in the engine repo alongside other tools.

Spatial-locality constraint on the data-region layout. Sibling tensors MUST be written physically adjacent in the `.sp-model` data region — specifically, `<weight>.scale` immediately follows `<weight>`, with no other tensor's bytes interposed. The transcoder sorts the data region in this order before writing:

1. Group tensors by their “base name” (everything before the final `.scale`, `.bias`, etc. suffix).
2. Within each group, write the parent first, then siblings in suffix-alphabetical order.
3. Write groups in topological order of access frequency (token embeddings first, then per-layer blocks, then output projection).

At inference time the kernel reads `weight` and immediately `weight.scale`. If they are physically separated by megabytes of unrelated tensors, `mmap` triggers two independent hard page faults and the OS prefetcher gets no hint that the second access is coming. If they are adjacent in the file (and therefore adjacent in the `mmap` region), the OS prefetcher pulls the scale page in with the weight page — a single 4 KB / 16 KB readahead window covers both. On a cold load this is the difference between $\sim 10\ \mu\text{s}$ and $\sim 150\ \mu\text{s}$ per Q8 layer's first decode step, multiplied across every layer in the model.

The constraint is the transcoder's responsibility; the loader does not need to validate it (the format is correct either way, just slower without the locality property). However, `sp-transcode --verify` should emit a warning if any sibling pair is non-adjacent, since that indicates a transcoder bug or hand-edited file.

10. Versioning and forward compatibility

- **v0** (this document): everything above.
- **v0.x** (no breaking change): new `dtype_id` values, new `arch_id` values. v0 readers refuse with `SP_EUNSUPPORTED` on unknown ids. Header bytes still occupy 512 bytes with `header_size == 512`.
- **v1** (potentially breaking): header may grow past 512 bytes. v0 readers refuse on `version_major != 0`. New fields go into the current reserved tail before growing the header.

Promotion criteria from v0 to v1: only when (a) the ABI requires a new field in `sp_arch_info` larger than the current 256-byte reservation, or (b) a structural property of the tensor data region changes (e.g. inline ARM bank initial state, KV warm-state snapshot). Neither is on the Phase-2 horizon.

11. Open questions / Phase-2+ considerations

- **Multi-file sharding for very large models.** A 200B model at Q4 is ~ 100 GB. A single `.sp-model` works on 64-bit filesystems but is unwieldy to distribute. v1 may add an optional shard-manifest sibling file (`.sp-shards`) pointing at multiple `.sp-model.NNNN-of-MMMM` parts. v0 assumes single file.
- **Direct DMA from .sp-model into GPU memory.** NVIDIA's GDS (GPUDirect Storage) reads from `O_DIRECT`-opened files into device memory without host buffering. Requires the tensor data region to be aligned to GPU page size (often 64 KB) and tensors to be at least 64 KB. Our existing 65536 alignment is already compatible; individual small tensors (norms, biases) below 64 KB still go through host buffering. Phase-2+ optimization.

- **In-file ARM bank seed.** Currently the ARM bank is initialized empty at session create. A pre-baked ARM bank (seeded with the golden-ratio key schedule from PPT-LAT-Systems §4.2) could ship as a regular tensor in `.sp-model`. v0 leaves the bank session-local; the spec accommodates this future addition without rev.
- **Tokenizer-blob compression.** SentencePiece blobs are usually small enough that compression is not worth it; HuggingFace `tokenizer.json` for some BPE-heavy tokenizers can hit 10 MB. v0 ships uncompressed; v1 may add a `blob_compression` field in the `.sp-tokenizer` header.

12. Sign-off checklist

Before this is locked alongside the ABI:

1. **mmap correctness.** Walk the load procedure in §8; confirm no step requires a malloc proportional to tensor data size. Each step is either a syscall, a memcpy of header-sized bytes, or a pointer assignment.
2. **Alignment table.** For every dtype in §5, confirm $(\text{tensor_data_offset} + \text{offset_in_data}) \% \text{required_align} == 0$ is enforceable by the transcoder. The two non-trivial cases are SPINOR63 (block_size 64, so the loader can stride by 64) and OK_Q4 (block_size 32, but offset_in_data must still be 64-aligned because we want SIMD on the first block).
3. **Round-trip.** A model transcoded GGUF $\rightarrow$ `.sp-model` $\rightarrow$ loaded by `sp_model_load` and run through `sp_prefill_chunk` produces logits bit-identical (deterministic mode) to running the original GGUF under `llama.cpp` with the same sampler off. This is the actual T_FRO_4-class gate for the format itself, separable from the ABI sign-off.

On green, both PPT-LAT-L1-ABI-v0 and PPT-LAT-SP-MODEL-v0 fold into PPT-LAT-Systems as Appendices A and B respectively, in one commit, and freeze together for Phase 2.

Status. v0 draft. Co-locked with PPT-LAT-L1-ABI-v0 once both have signed off against their §12 checklists.